



MÔN HỌC: MÁY HỌC TRONG BẢO MẬT MẠNG VÀ HỆ THỐNG

HỆ THỐNG LLM ĐA TÁC TỬ Y KHOA NHẪM NÂNG CAO ĐỘ CHÍNH XÁC THỰC TẾ TRÊN MEDQA-USMLE

Báo cáo giữa kỳ

Giảng viên hướng dẫn: Lê Kim Hùng

Nhóm thực hiện: Nhóm 01

Thành viên:

- Nguyễn Đình Thanh — [CẦN LÀM: MSSV]
- Ngô Văn Tú — [CẦN LÀM: MSSV]
- Lê Viết Hưng — [CẦN LÀM: MSSV]
- Trần Quốc Phong — [CẦN LÀM: MSSV]

Mục lục

1	Giới thiệu	2
2	Kiến trúc hệ thống	3
3	Thiết kế hệ thống	6
3.1	Các biến thể: V0–V4	6
3.2	Thiết kế RAG	7
3.3	Thiết kế bộ nhớ	11
3.4	Thiết kế bộ kiểm chứng	12
3.5	Tối ưu hoá tiên hoá	12
4	Thiết lập thực nghiệm	12
4.1	Mô hình và prompt	12
4.2	Cấu hình truy hồi	13
4.3	Phần cứng / thiết lập API	14
5	Kết quả đánh giá	14
5.1	Bảng chính (main board): test đầy đủ ($n=1273$, qwen2.5:7b)	14
5.2	So sánh với baseline và khảo sát ablation (so sánh ghép cặp)	15
5.3	Phân tích Thắng/Thua/Hòa	15
5.4	Chi phí và độ trễ	15
6	Phân tích lỗi	15
7	Thảo luận	16
8	Hạn chế	16
9	Kết luận	17
A	Danh mục kiểm tra tái lập	17

Tóm tắt

Nhóm xây dựng và đánh giá một hệ thống LLM đa tác tử (multi-agent) y khoa dạng module với mục tiêu nâng cao *độ chính xác thực tế* (factual accuracy) trên bộ dữ liệu MedQA-USMLE. Hệ thống được tổ chức thành một **thang biến thể** (ablation ladder) gồm năm biến thể (V0 LLM trực tiếp $\rightarrow$ V4 hệ đa tác tử đầy đủ); mỗi biến thể chỉ là cùng một sơ đồ xử lý (StateGraph) của LangGraph, nhưng bật/tắt vài thành phần theo một cấu hình định sẵn tương ứng. Nhờ hai biến thể liền nhau chỉ khác đúng một thành phần, mọi chênh lệch về độ chính xác đều quy thẳng được về thành phần đó. Tất cả biến thể dùng chung một giao diện thống nhất và một bộ đo (độ chính xác kèm khoảng tin cậy bootstrap, tỷ lệ phản hồi không hợp lệ, phân tích Thắng/Thua/Hòa, và kiểm định ghép cặp McNemar), giúp phép so sánh trung thực. Phát hiện thực nghiệm cốt lõi của Nhóm là: trên MedQA **riêng truy hồi (retrieval) không giúp ích** ($V1 \leq V0$), trong khi **lập luận đa tác tử mới là bước cải thiện thực sự đầu tiên** (kết quả định lượng đang được đo lại trên test đầy đủ). Nhóm trình bày kết quả đo hiện tại, ghi lại đầy đủ kiến trúc, prompt và cấu hình truy hồi để tái lập được, đồng thời liệt kê các phần việc còn lại để đạt tới đánh giá chính thức đầy đủ.

1 Giới thiệu

Các mô hình ngôn ngữ lớn (LLM) trả lời đúng nhiều câu hỏi thi y khoa ngay từ đầu, nhưng chúng cũng đưa ra những câu trả lời sai một cách rất tự tin (nghe thuyết phục nhưng sai), và không dễ thấy *thành phần kiến trúc nào* thực sự làm giảm những lỗi đó. Dự án này đặt một câu hỏi về thiết kế chứ không phải triển khai lâm sàng: bắt đầu từ một LLM thuần, những thành phần nào giúp cải thiện đo lường được độ chính xác thực tế trên MedQA-USMLE, và cải thiện bao nhiêu? Cụ thể là ba thành phần: truy hồi tri thức (RAG), phối hợp nhiều tác tử (multi-agent), và một bước kiểm chứng (verifier).

Bài toán. Đầu vào là một câu hỏi trắc nghiệm MedQA-USMLE [1] (một tình huống lâm sàng kèm bốn lựa chọn A–D). Đầu ra bắt buộc là *đúng một* lựa chọn cuối cùng cộng với một giải thích ngắn (≤ 30 từ).

Cách tiếp cận. Nhóm không viết tay năm hệ thống riêng biệt. Thay vào đó, mỗi biến thể là *cùng một* bộ khung LangGraph, chỉ bật/tắt các nút (node) khác nhau theo một cấu hình định sẵn tương ứng. Cách này giúp so sánh mang tính nhân quả: giữa V0 và V1 chỉ khác các nút truy hồi, giữa V2 và V3 chỉ khác cụm hội chẩn / bác sĩ chính / kiểm chứng. Và vì chỉ một bộ đo chấm cả năm biến thể theo cùng một cách, mọi chênh lệch điểm số đều so sánh được trực tiếp.

Đóng góp.

1. Một kiến trúc đa tác tử điều khiển bằng cấu hình (V0–V4), trong đó mỗi bậc thang khác bậc kế tiếp đúng một thành phần (Mục 2–3).
2. Một quy trình đánh giá ghép cặp, tái lập được, có kiểm định ý nghĩa thống kê (Mục 4–5).
3. Kết quả thực nghiệm cho thấy trên MedQA, *cấu trúc lập luận*, chứ không phải *truy hồi*, mới là đòn bẩy làm thay đổi độ chính xác (Mục 5–7).

Trạng thái đo lường (đọc phần này trước). Kiến trúc (V0–V4) đã được xây dựng đầy đủ và kiểm tra chạy được đầu-cuối, nhưng đã trải qua một đợt tái thiết kế sau lần đo gần nhất: V2–V4 đổi từ thiết kế hội đồng nhiều chuyên gia sang thiết kế bốn nút hiện tại (§3), và V1–V4 nay dùng chung một luật quyết định mới thêm vào — *tin cậy bất đối xứng*. Vì vậy các số đo V1/V2 ở §5 mô

tả thiết kế trước đó, không phải kiến trúc hiện tại, và cần đo lại; chỉ V0 (không dùng truy hồi hay tác tử thứ hai, nên không bị đột tái thiết kế này ảnh hưởng) còn giữ nguyên giá trị tham chiếu. Mỗi khoảng trống như vậy đều được đánh dấu **[CẦN LÀM:]** để người đọc biết chính xác điều gì đã đo và điều gì mới là dự kiến.

2 Kiến trúc hệ thống

Ý tưởng tổ chức là một bộ khung đồ thị duy nhất với các nút được bật/tắt tùy biến thể:

$$[\text{understand}] \rightarrow (\text{search} \parallel \text{reasoning}) \rightarrow \text{decide} \rightarrow [\text{verify}]$$

Một biến thể không phải một nhánh mã riêng; nó là một `RunConfig` định sẵn được biên dịch thành một `StateGraph` của `LangGraph` [4] qua `build_graph(cfg)`. Chính điều này khiến khảo sát ablation trung thực.

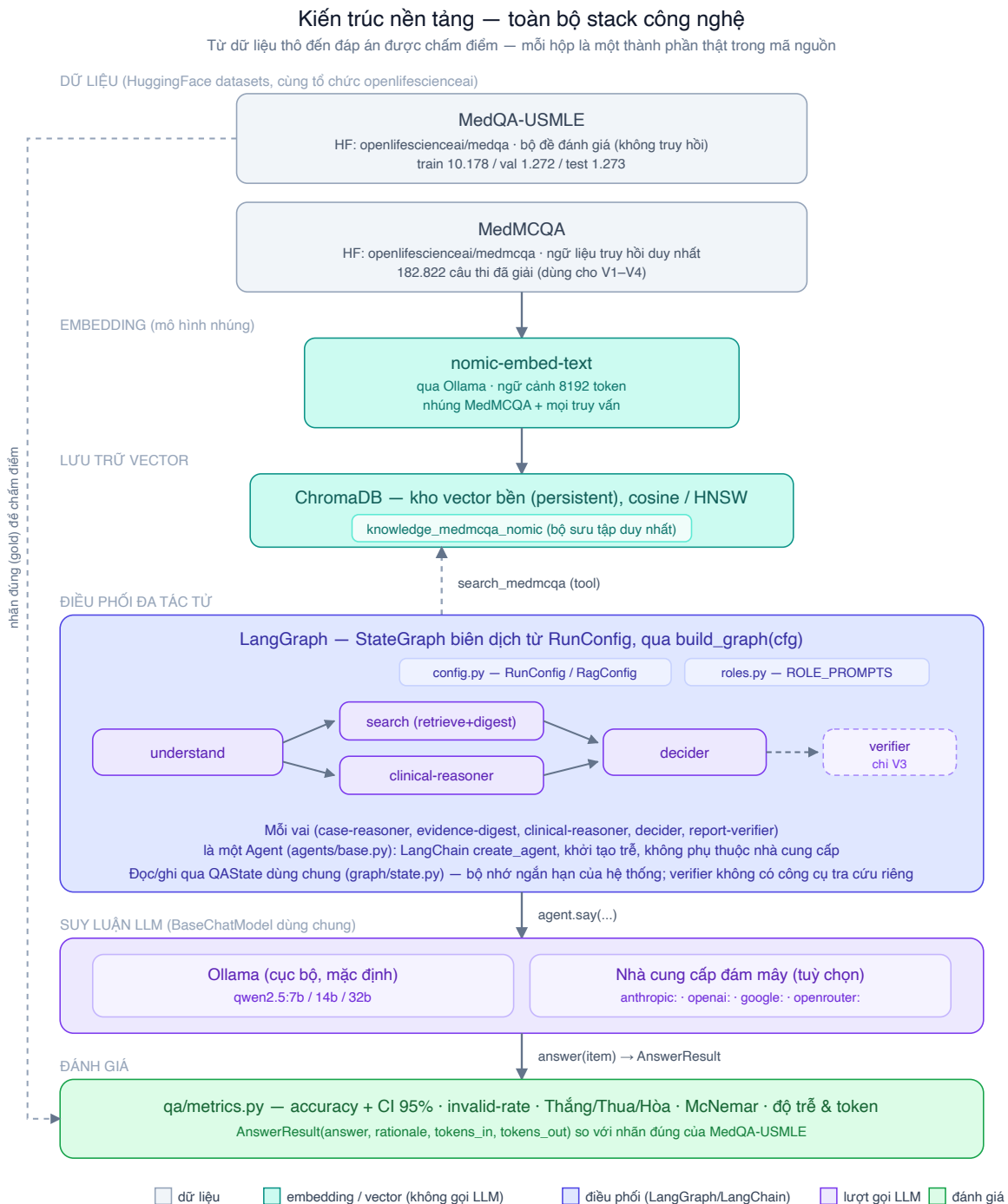
Vai trò từng thành phần trong stack (Hình 1).

- **Dữ liệu (HuggingFace datasets):** **MedQA-USMLE** [1] là bộ đề dùng để đánh giá, không bao giờ dùng để truy hồi; **MedMCQA** [7] là ngữ liệu truy hồi duy nhất, dùng chung cho toàn bộ V1–V4 (kể cả bộ kiểm chứng của V3, §3.4 — không còn nguồn thứ hai).
- **Embedding: nomic-embed-text** (chạy qua Ollama, ngữ cảnh 8192 token) nhúng MedMCQA và mọi truy vấn, cho mọi biến thể.
- **Lưu trữ vector — ChromaDB:** kho vector bền (persistent), chỉ mục cosine/HNSW, một bộ sưu tập duy nhất (`knowledge_medmcqa_nomic`).
- **Điều phối đa tác tử — LangGraph:** `StateGraph` biên dịch một `RunConfig` (`config.py`) thành đồ thị các nút (`graph/nodes.py`, `graph/build.py`); mọi nút đọc/ghi qua `QAState` dùng chung (`graph/state.py`) — đây chính là bộ nhớ ngắn hạn của hệ thống (§3.3).
- **Tác tử — LangChain:** mỗi vai trong đồ thị (5 vai, xem Hình 1) là một **Agent** (`agents/base.py`), bọc `create_agent` của `LangChain` — khởi tạo trễ, không phụ thuộc nhà cung cấp. Prompt của từng vai nằm trong `roles.py`.
- **Suy luận LLM:** Ollama cục bộ (qwen2.5, cỡ 7b/14b/32b) là mặc định; các nhà cung cấp đám mây (`anthropic:`, `openai:`, `google:`, `openrouter:`) cắm vào cùng giao diện `BaseChatModel` qua một chuỗi `provider:model` duy nhất (§4).
- **Đánh giá:** `qa/metrics.py` so `AnswerResult` (đáp án, giải thích, token) với nhãn đúng của MedQA-USMLE, tính độ chính xác kèm khoảng tin cậy bootstrap, tỷ lệ không hợp lệ, Thắng/Thua/Hòa, và McNemar (§5).

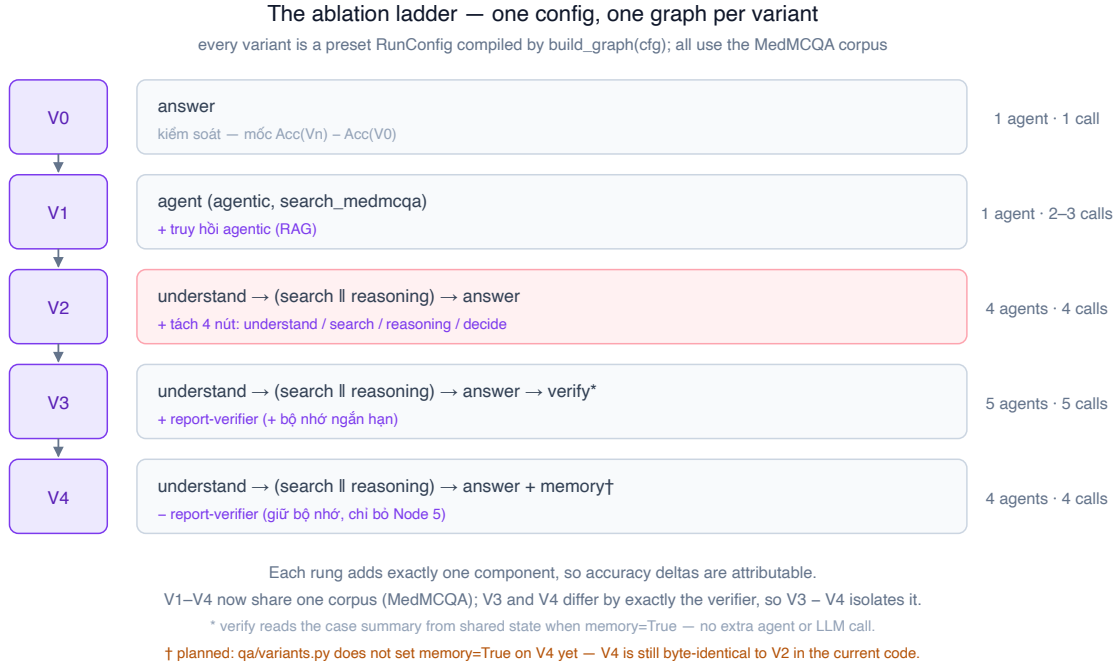
Bảng 1: Các nút thực sự được nối trong mỗi biến thể (tên nút khớp với mã nguồn).

Biến thể	Các nút được nối	Tác tử (lượt gọi LLM)
V0	<code>answer</code>	1
V1	<code>agent</code> (agentic, gọi <code>search_medmcqa</code>)	1
V2	<code>understand</code> → (<code>search</code> ∥ <code>reasoning</code>) → <code>answer</code>	4
V3	<code>understand</code> → (<code>search</code> ∥ <code>reasoning</code>) → <code>answer</code> → <code>verify</code> + <code>memory</code>	5
V4	<code>understand</code> → (<code>search</code> ∥ <code>reasoning</code>) → <code>answer</code>	4

V1 là ngoại lệ của bộ khung trên: thay vì tách `understand/search/reasoning` thành các nút riêng, nó rút về một nút `agent` duy nhất, tự làm cả bốn việc trong một lượt hội thoại có gọi công



Hình 1: Toàn bộ stack công nghệ, từ dữ liệu thô đến đáp án được chấm điểm. Mỗi hộp khớp với một thành phần thật trong mã nguồn (tên tệp/hàm ghi kèm).



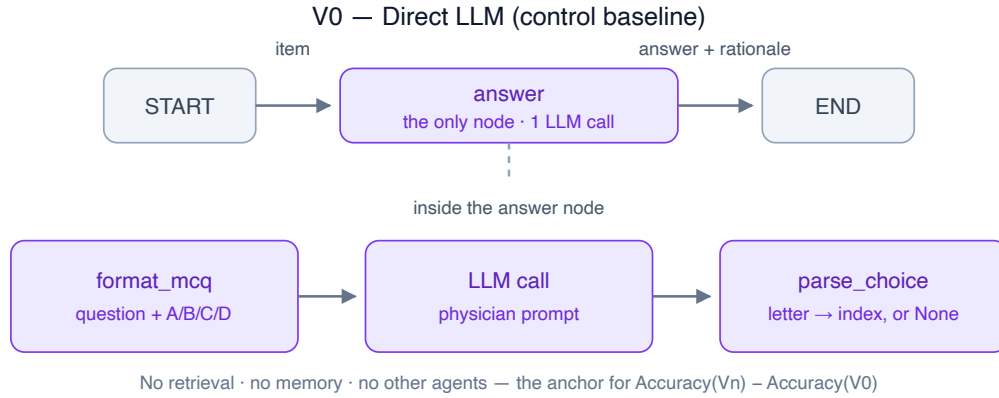
Hình 2: Thang khảo sát ablation. Mỗi biến thể thêm đúng một khả năng so với biến thể trước.

cụ (xem §3.2). V2–V4 theo đúng bộ khung bốn nút, với Node 1 (**understand**) sinh ra bản tóm tắt ca bệnh + truy vấn dùng chung cho hai nhánh Node 2 (**search**) và Node 3 (**reasoning**) chạy *đồng thời*, trước khi Node 4 (**answer**, vai **decider**) tổng hợp; V3 thêm Node 5 (**verify**).

Bản đồ module. Toàn hệ thống nằm gọn trong một số ít tệp, mỗi tệp một nhiệm vụ: `config.py` (nơi tập trung mọi tùy chỉnh: `RunConfig/RagConfig`), `roles.py` (mọi system prompt), `graph/` (trạng thái, các hàm tạo node, và `build_graph`), `qa/variants.py` (các cấu hình định sẵn V0–V4 và điểm vào công khai `build_variant`), `qa/mcq.py` (định dạng prompt và trích xuất câu trả lời), `qa/metrics.py` (mọi phép đo), `knowledge/` (nạp dữ liệu RAG, embedding, bộ truy hồi), và `agents/base.py` (lớp bọc tác tử khởi tạo sẵn, không phụ thuộc nhà cung cấp).

Các tính chất thiết kế.

- **Giao diện thống nhất:** mọi biến thể đều là `answer(item) → AnswerResult(answer, rationale)`, nên một bộ đo duy nhất chấm điểm tất cả như nhau.
- **Khởi tạo sẵn / an toàn khi offline:** tác tử được dựng ở lần gọi đầu và kho vector mở ở lần truy hồi đầu, nên việc biên dịch một đồ thị không dùng tới mạng; các bài kiểm thử chạy ngoại tuyến.
- **Mỗi lần một thứ:** thêm một biến thể hay A/B một tham số (mô hình, embedder, kích thước hội đồng, bật/tắt kiểm chứng) là thay đổi cấu hình chứ không phải viết mã mới.



Hình 3: V0 — một node duy nhất, không truy hồi, không tác tử thứ hai.

3 Thiết kế hệ thống

3.1 Các biến thể: V0–V4

Mỗi tác tử là một lớp bọc mỏng, khởi tạo trể quanh `create_agent` của LangChain với một `BaseChatModel` không phụ thuộc nhà cung cấp; chỉ cần ghi tên mô hình dưới dạng chuỗi là nó được hiểu thành một mô hình Ollama cục bộ. Mục này giới thiệu từng biến thể theo đúng thứ tự thang ablation — luồng xử lý, các tác tử tham gia, và điều mỗi bậc thêm vào so với bậc trước; §3.2 đi sâu hơn vào các quyết định thiết kế truy hồi dùng chung cho V1–V4.

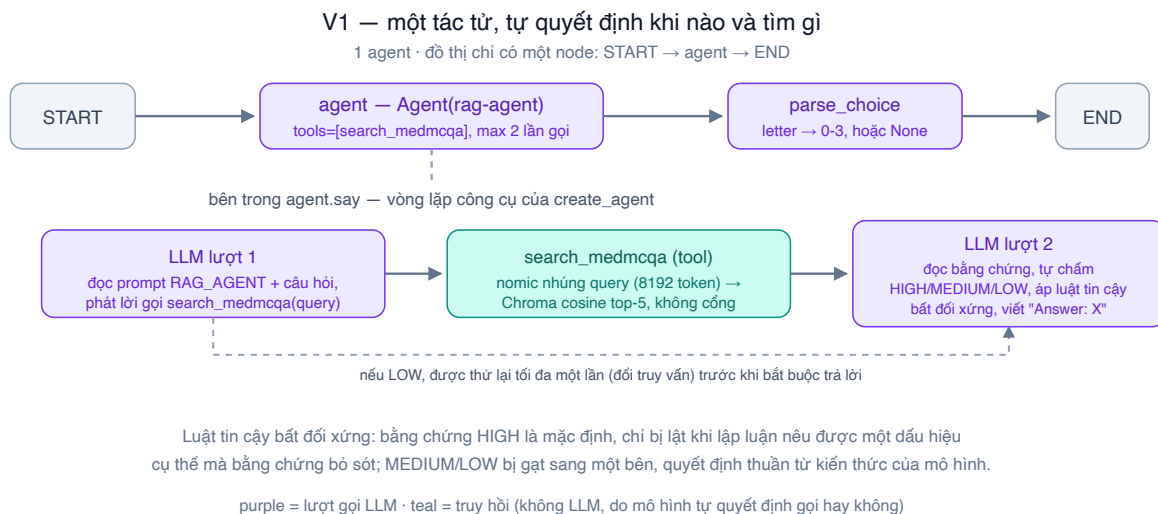
V0 — LLM trực tiếp (đối chứng). Một lượt gọi LLM duy nhất, không truy hồi, không tác tử thứ hai: `format_mcq` dựng prompt (tình huống + A–D), model trả lời, `parse_choice` trích chữ cái. Đây là mốc mà mọi biến thể khác được so sánh vào: Độ chính xác(Vn) – Độ chính xác(V0).

V1 — một tác tử agentic RAG. Đồ thị rút về một node: `START` → `agent` → `END`. Bên trong, `Agent(rag-agent)` tự quyết định có gọi `search_medmcqa` hay không (tối đa hai lần), tự chấm độ tin cậy của bằng chứng lấy về (HIGH/MEDIUM/LOW), rồi áp luật *tin cậy bất đối xứng* (§3.2) để chốt đáp án — tất cả trong cùng một tác tử, không có node/tác tử riêng cho từng bước.

V2 — bốn nút: hiểu ca bệnh, tìm kiếm, lập luận, quyết định. V2–V4 dùng chung một thiết kế bốn nút, tách đúng bốn việc mà tác tử đơn của V1 làm nối tiếp bên trong nó thành bốn nút riêng, chạy qua LangGraph thay vì một vòng lặp công cụ:

- **Node 1 — case-reasoner (understand):** đọc tình huống một lần, viết ra một bản tóm tắt ca bệnh và một truy vấn tìm kiếm, dùng chung cho hai nhánh sau; không nêu đáp án.
- **Node 2 — search (tìm kiếm + rút gọn):** gọi `search_medmcqa` với truy vấn của Node 1, rồi `evidence-digest` tóm bằng chứng thành một đoạn ngắn kèm mức tin cậy tự đánh giá.
- **Node 3 — clinical-reasoner (reasoning):** lập luận từ CÙNG bản tóm tắt của Node 1, hoàn toàn không thấy bằng chứng của Node 2 (hai nhánh chạy đồng thời); tự chấm mức tin cậy của chính lập luận đó.
- **Node 4 — decider (answer):** nhận hai đầu vào độc lập kèm mức tin cậy riêng, áp luật tin cậy bất đối xứng để chốt đáp án.

4 tác tử, 4 lượt gọi LLM/câu, không có bộ kiểm chứng. Thiết kế này khác thiết kế hội đồng nhiều chuyên gia (kiểu MedAgents [9]) mà Nhóm đã thử trước đó: thay vì nhiều tác tử tranh luận theo



Hình 4: V1 — một tác tử agentic tự gọi công cụ, tự chấm độ tin cậy, tự áp luật quyết định.

nhiều góc nhìn rồi một bác sĩ chính hoà giải, Node 2 và Node 3 tách biệt hoàn toàn, và Node 4 dùng một luật tường minh để quyết nhánh nào được tin.

V3 — V2 + bộ kiểm chứng + bộ nhớ ngắn hạn. V3 thêm **Node 5 — report-verifier**: đọc lại báo cáo lập luận của Node 3, và, khi bật bộ nhớ ngắn hạn (**memory=True** — lần đầu xuất hiện ở biến thể này, xem §3.3), cả bản tóm tắt ca bệnh của Node 1, thẳng từ trạng thái đồ thị dùng chung (không qua một tác tử/lượt gọi LLM riêng), rồi xác nhận hoặc chỉnh đáp án của Node 4 (§3.4). 5 tác tử, 5 lượt gọi LLM/câu.

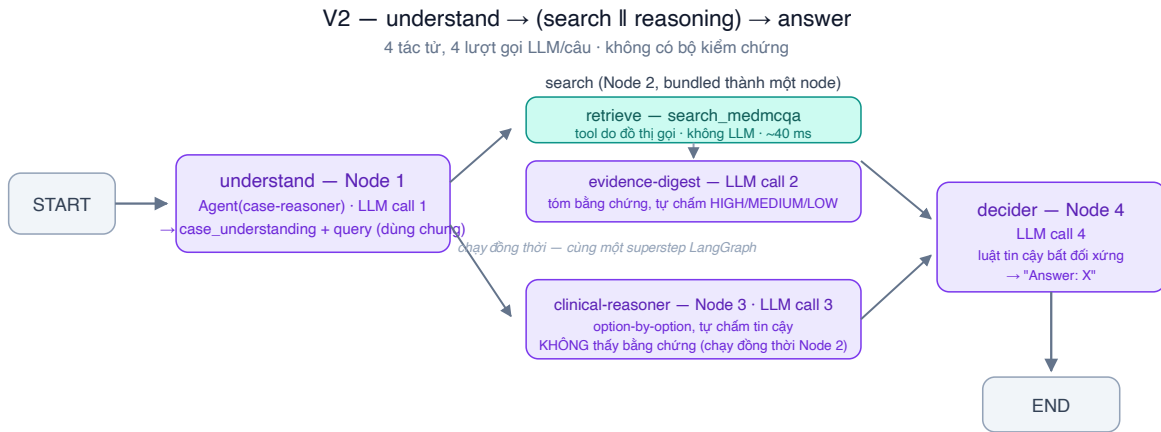
V4 — V2 không có bộ kiểm chứng. V4 hiện là *đúng cấu hình của V2* (**verify=False**, **memory=False**) chứ không phải một preset kiến trúc riêng — nhờ vậy V3 — V4 chỉ khác đúng Node 5, không lẫn với ngữ liệu/embedder/số tác tử như hạn chế đã nêu trong bản báo cáo trước (§3.4). **[CẦN LÀM: Kế hoạch: bật memory=True cho V4 để V3/V4 chỉ khác đúng field verify, thay vì hai field cùng là no-op khi không có Node 5; qa/variants.py chưa cập nhật.]**

3.2 Thiết kế RAG

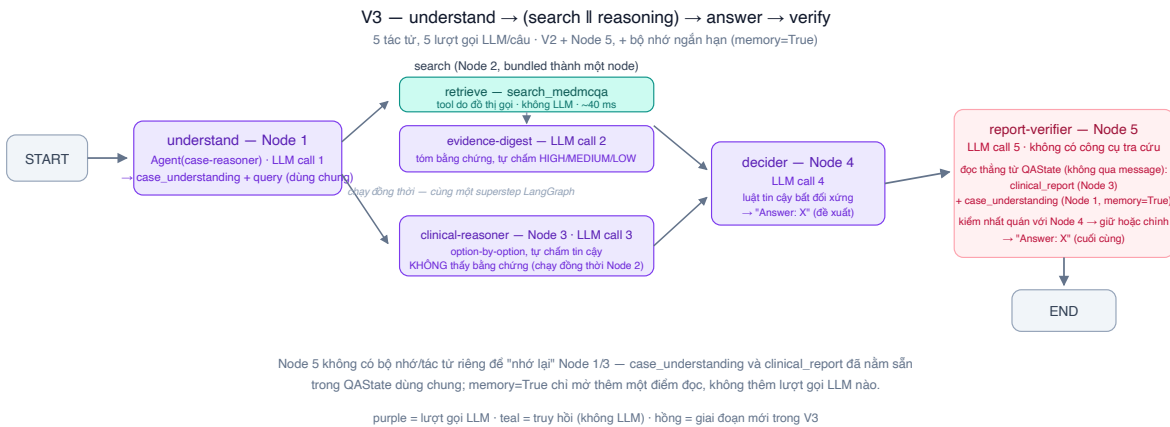
V1–V4 nay dùng chung một nguồn truy hồi: **MedMCQA** (182.822 câu thi đã giải), nhúng bằng **nomic-embed-text** (ngữ cảnh 8192 token, nên không bị cắt như Query encoder 64 token của MedCPT). Đây là một thay đổi so với thiết kế trước, khi V1/V2 tra MedMCQA còn V3/V4 tra sách giáo khoa (MedRAG Textbooks) — một biến gây nhiễu giữa ngữ liệu và kiến trúc mà bản báo cáo trước liệt kê là hạn chế; hợp nhất ngữ liệu như hiện tại giải quyết hạn chế đó (chênh lệch giữa các bậc nay chỉ còn do kiến trúc, không lẫn với việc đổi ngữ liệu giữa các bậc nữa).

Cách mỗi biến thể dùng ngữ liệu đó thì khác nhau.

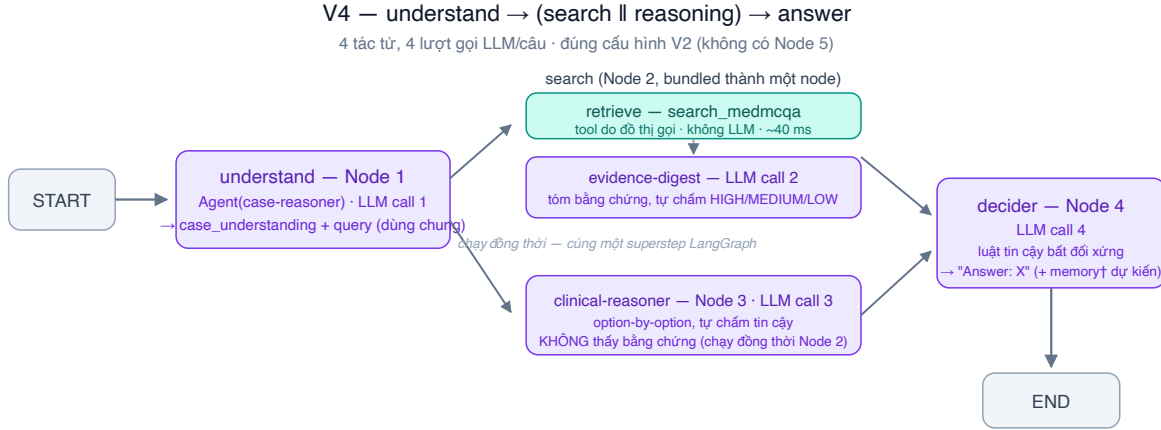
1. **V1 — một tác tử tự quyết định khi nào và tìm gì.** Không có pipeline cố định: một tác tử duy nhất tự viết truy vấn, tự gọi `search_medmcqa` (tối đa hai lần — được phép thử lại



Hình 5: V2 — Node 1 phân nhánh đồng thời sang Node 2/Node 3, hội tụ ở Node 4.



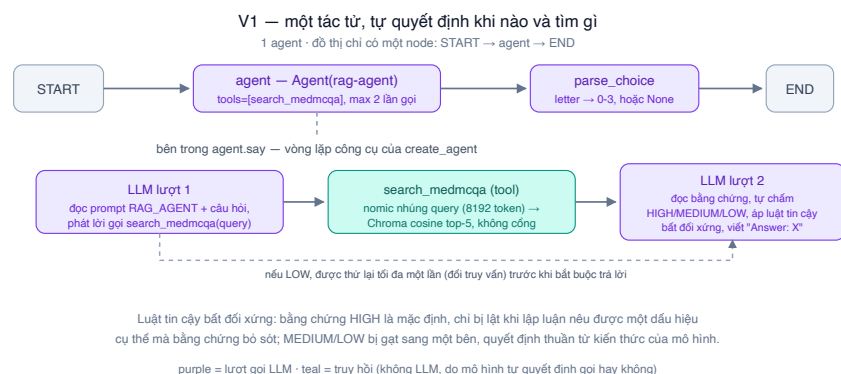
Hình 6: V3 — Node 5 audit đáp án của Node 4, đọc thẳng Node 1/Node 3 từ QASate dùng chung.



Hình 7: V4 — cùng đồ thị với V2 (Hình 5); không có Node 5.

- một lần nếu kết quả đầu có vẻ không sát), đọc kết quả, tự chấm độ tin cậy của bằng chứng đó rồi quyết định. Đồ thị rút về một nút: **START** → **agent** → **END**.
- V2–V4 — tách tìm-kiểm khỏi lập-luận, chạy đồng thời.** Node 1 (**understand**) viết một bản tóm tắt ca bệnh và một truy vấn dùng chung; Node 2 (**search**) gọi `search_medmcqa` với truy vấn đó — do *đô thị* gọi trực tiếp, không phải mô hình tự chọn gọi hay không, nên xác định được (deterministic) và không tốn thêm lượt LLM chỉ để gọi công cụ — rồi một tác tử **evidence-digest** tóm bằng chứng lấy về thành một đoạn ngắn kèm mức tin cậy tự chấm. Node 3 (**reasoning**) lập luận lâm sàng từ CÙNG bản tóm tắt của Node 1, không thấy bằng chứng của Node 2 (hai nhánh chạy đồng thời), và cũng tự chấm độ tin cậy của lập luận đó.
 - Công cụ truy hồi top- k , không còn cổng số.** `search_medmcqa` luôn trả về $k=5$ câu thi gần nhất kèm đáp án và giải thích (định dạng Q:/A:/Why:), cho mọi biến thể (`threshold=0.0`). Thiết kế trước dùng một *cổng độ tương đồng* bằng số (ngưỡng cosine, hiệu chỉnh riêng theo từng embedder); cổng đó nay được thay bằng **mô hình tự đánh giá**: bước chốt đáp án dùng nhãn tin cậy tự chấm (HIGH/MEDIUM/LOW) của chính tác tử/nút vừa đọc bằng chứng, chứ không so một điểm cosine với một ngưỡng cố định.
 - Luật quyết định: tin cậy bất đối xứng.** Cùng một luật cho cả bước chốt đáp án của V1 lẫn Node 4 (**decider**, V2–V4): nếu bằng chứng truy hồi được chấm HIGH và rõ ràng nghiêng về một phương án, phương án đó là đáp án mặc định — chỉ bị lật lại khi có một dấu hiệu *cụ thể* trong tình huống mà bằng chứng đã bỏ sót hoặc hiểu sai, không phải một cảm giác chung chung là “có lẽ khác”. Nếu bằng chứng chỉ MEDIUM/LOW hoặc không có, nó bị gạt sang một bên hoàn toàn và đáp án được chọn thuần từ lập luận lâm sàng, đúng như cách V0 quyết định. Luật bất đối xứng có chủ đích: bằng chứng tốt được ưu tiên mặc định, nhưng không tự động thắng một lập luận nên được dấu hiệu cụ thể.

Truy hồi là công cụ do mô hình chọn, chỉ ở V1. V1 để tác tử tự quyết định khi nào và tìm gì, thay vì *đô thị* gọi cứng như V2–V4. Đây là chi phí có chủ đích: đo trên cùng tập câu, cách đo tác tử điều khiển tốn 3 lần gọi LLM / 542 ký tự sinh ra / 3.78s mỗi câu, so với 2 lần gọi / 51 ký



Hình 8: Luồng RAG của V1 (agentic): một tác tử duy nhất tự gọi công cụ `search_medmcqa`, tự chấm độ tin cậy của bằng chứng rồi áp luật tin cậy bất đối xứng. **[CẦN LÀM: Vẽ lại v1_flow.svg: hình hiện tại chưa thể hiện bước tự chấm độ tin cậy / luật quyết định.]**

tự / 0.90 s khi để đồ thị gọi: một lời gọi công cụ do mô hình chọn cần thêm một lần gọi để xử lý kết quả, và tác tử dùng công cụ không thể bị ép “chỉ trả lời bằng chữ cái” (chỉ dẫn đó triệt tiêu việc gọi công cụ; đo được 0/5), nên đáp án được trích từ phần lý do ngắn thay vì một chữ cái đơn. Nhóm chấp nhận chi phí này vì mục tiêu là V1 thành một tác tử RAG đúng nghĩa; V2–V4 vẫn gọi truy hồi từ node để giữ tính tất định và chi phí thấp.

Luồng mã của V1. Từ `build_variant("V1")` đến đáp án, đường đi cụ thể như sau (tên tệp ghi bên phải):

```
build_variant("V1")                                     # qa/variants.py
  preset: answer_role="rag-agent",
          rag=(knowledge_medmcqa_nomic, nomic, k=5, threshold=0.0, tool=True)
  build_graph(cfg) -> đồ thị 1 node: START -> agent -> END    # graph/build.py

answer(item) = graph.invoke({"item": item})              # qa/variants.py

[node] make_agentic_rag_node                             # graph/nodes.py
  tool = search_medmcqa (make_medmcqa_tool, max_calls=2)
  agent = Agent("rag-agent", tools=[tool])
  reply = agent.say( format_mcq(item, AGENTIC_ANSWER) )
  return answer = parse_choice(reply), rationale = reply

trong agent.say (vòng lặp công cụ của create_agent): # agents/base.py
  LLM lượt 1: he thông=RAG_AGENT + MCQ + mô tả tool
  -> phát một lời gọi công cụ: search_medmcqa(query)
      search_medmcqa(query):                               # graph/nodes.py
        retrieve(query, k=5, threshold=0.0)                # knowledge/retriever.py
        nomic nhúng query (8192 token, không cắt)
        Chroma cosine top-k, khu trung lặp, KHÔNG cổng
        format_evidence(hits) -> "Q:/A:/Why:" x5
        <- kết quả tool được nối vào chuỗi message
  LLM lượt 2: đọc bằng chứng, tự chấm HIGH/MEDIUM/LOW, áp luật tin cậy bất đối xứng,
              viết "... Answer: X"
  parse_choice(reply) -> 0..3 (hoặc None)                  # qa/mcq.py
```

Hai điểm mấu chốt: (1) *mô hình* tự quyết định gọi công cụ, nên đồ thị chỉ có một node; (2) truy

hỏi trả về top- k không qua cổng số, và bước quyết định ở lượt 2 dùng nhãn tin cậy tự chấm thay cho một ngưỡng cosine. Chi phí ~ 2 lần sinh của LLM mỗi câu (lượt 1 gọi công cụ, lượt 2 trả lời) là lý do V1 chậm hơn hẳn V0.

Luồng mã của V2. V2 tách bốn việc mà tác tử đơn của V1 làm nối tiếp (hiểu ca bệnh, tìm kiếm, lập luận, quyết định) thành bốn nút, hai trong số đó (**search**, **reasoning**) chạy đồng thời:

```
V2:  START -> understand -> {search, reasoning} -> answer -> END
      understand: Agent("case-reasoner")    # graph/nodes.py:make_understand_node
              -> case_understanding (tom tat ca benh) + query (dung chung)
      search:    retrieve(query) -> Agent("evidence-digest")    # make_search_branch_node
              -> evidence = tom bang chung + muc tin cay tu cham
      reasoning: Agent("clinical-reasoner"), doc case_understanding, KHONG thay evidence
              -> clinical_report = lap luan tung phuong an + muc tin cay tu cham
      answer:    Agent("decider"), doc evidence VA clinical_report    # make_answer_node
              -> ap luat tin cay bat doi xung -> "... Answer: X"
```

search và **reasoning** được nối trực tiếp từ **understand** (không nối tiếp nhau), nên LangGraph chạy chúng trong cùng một superstep; **answer** chờ CẢ HAI xong mới chạy (nối bằng `add_edge([search, reasoning], answer)`, dạng danh sách, để **answer** chỉ chạy đúng một lần). **4 tác tử, 4 lượt gọi LLM/câu** (**understand**, **evidence-digest**, **clinical-reasoner**, **decider**) — không còn hội đồng nhiều chuyên gia tranh luận theo góc nhìn (thiết kế trước, kiểu MedAgents [9]); thay vào đó hai nhánh độc lập KHÔNG thấy đầu ra của nhau, và **decider** dùng một luật tường minh (chứ không phải một bước tổng hợp tự do) để quyết nhánh nào được tin. **[CẦN LÀM: Do V2 mới ở quy mô lớn; số cũ ở §5 do trên thiết kế hội đồng trước đó, không áp dụng được cho thiết kế này.]**

Lựa chọn ngữ liệu. MedMCQA (Pal et al. [7], 182.822 câu thi đã giải kèm giải thích) là ngữ liệu truy hồi *duy nhất* trong hệ thống hiện tại, dùng cho toàn bộ V1–V4 — tức truy hồi các ví dụ mẫu đã giải (tương tự Medprompt [6]), không phải văn bản tham khảo. Bộ kiểm chứng của V3 (§3.4) không còn công cụ tra cứu riêng: một bản thiết kế trước đó cho nó một nguồn thứ hai, độc lập (MedRAG Textbooks, nhúng bằng MedCPT) để đối chiếu đáp án đề xuất với văn bản tham chiếu; nguồn đó đã bị bỏ, và bộ kiểm chứng nay chỉ audit dựa trên báo cáo lập luận đã có trong trạng thái đồ thị dùng chung (§3.4).

3.3 Thiết kế bộ nhớ

Đề bài liệt kê bộ nhớ ngắn hạn và dài hạn là các thành phần bắt buộc. **Bộ nhớ ngắn hạn: đã xây, ở dạng hẹp.** Thay vì một tác tử/kho bộ nhớ riêng, bộ nhớ ngắn hạn tận dụng `QASState` — trạng thái dùng chung mà LangGraph truyền qua các nút: **case_understanding** (bản tóm tắt ca bệnh của Node 1), **evidence**, và **clinical_report** (báo cáo của Node 3) đều đã nằm sẵn trong đó. Khi `memory=True` (V3), bộ kiểm chứng (Node 5) đọc thêm **case_understanding** khi audit đáp án, thay vì chỉ đọc báo cáo lập luận như V2/V4; không nút nào khác cần bật cờ này, vì các nút khác vốn đã đọc thẳng những gì nút trước ghi ra — `memory` chỉ mở rộng đúng MỘT điểm đọc thêm cho bộ kiểm chứng, không thêm tác tử hay lượt gọi LLM nào. **Bộ nhớ dài hạn (episodic, xuyên câu hỏi): chưa xây**, và không nằm trong bất kỳ biến thể nào hiện có. Một kế hoạch trước đó đã hình dung việc truy hồi lập luận từ các lỗi đã giải trong tập MedQA-train; điều đó vẫn nằm ngoài phạm vi và là việc trong tương lai. **[CẦN LÀM: Xây bộ nhớ dài hạn xuyên câu hỏi; do đó hệ thống hiện tại vẫn chưa phải “hệ đầy đủ” theo đúng nghĩa đề bài đòi bộ nhớ dài hạn.]**

3.4 Thiết kế bộ kiểm chứng

V3 thêm Node 5 (`report-verifier`) làm khâu cuối: nó đọc thẳng báo cáo lập luận của Node 3 (`state["clinical_report"]`), đáp án và giải thích ngắn mà Node 4 vừa chốt, và, khi bật bộ nhớ (§3.3), cả bản tóm tắt ca bệnh của Node 1 — không cần một tác tử/lượt gọi LLM riêng để “nhớ lại” những thứ đó, vì chúng đã nằm sẵn trong trạng thái đồ thị dùng chung. Nó chỉ kiểm nhất quán: nếu đáp án được chọn bị chính báo cáo lập luận đánh giá RULED OUT (hoặc một phương án khác được đánh giá rõ SUPPORTED), đó là dấu hiệu đáng ngờ và nó chỉnh lại đáp án cho khớp với những gì báo cáo đã kết luận. **Không còn công cụ tra cứu riêng:** một bản thiết kế trước đó cho Node 5 một `search_textbooks` đọc lập trên MedRAG Textbooks (nhúng bằng MedCPT) để đối chiếu với văn bản tham chiếu khi báo cáo không đủ để quyết; nguồn thứ hai đó đã bị bỏ — bộ kiểm chứng nay chỉ audit dựa trên bằng chứng và lập luận Node 2/Node 3 đã tạo ra, không tự truy hồi thêm gì. Để giữ luồng bền vững, một lỗi trích xuất ở Node 5 sẽ lùi về đáp án của Node 4 thay vì phát ra `None` không hợp lệ.

Phần đóng góp riêng của bộ kiểm chứng được đo trực tiếp bằng tương phản V3 – V4: vì V4 nay là *đúng cấu hình của V2* (không có Node 5, không bật bộ nhớ) chứ không còn là một preset kiến trúc cũ như bản báo cáo trước, tương phản này thực sự chỉ khác đúng một thứ — Node 5 (kèm quyền đọc bộ nhớ) — không còn lẫn với ngữ liệu, embedder, hay số tác tử của panel như hạn chế đã nêu trong bản báo cáo trước; hạn chế đó nay coi như đã được giải quyết. **[CẦN LÀM: Báo cáo V3 – V4 khi cả hai được đo ở quy mô lớn.]**

3.5 Tối ưu hoá tiến hoá

Không dùng trong hệ giữa kỳ này. Đề bài liệt kê nó là tùy chọn; Nhóm chủ động bỏ qua để giữ khảo sát ablation dễ diễn giải và dời việc tìm kiếm prompt/pipeline sang phần sau.

Đáp án + giải thích. Đề bài yêu cầu một lựa chọn *cộng với* một giải thích ngắn. Mọi biến thể đều trả về cả hai qua một giao diện duy nhất `answer(item) → AnswerResult(answer, rationale)`, và bản ghi từng câu mang theo phần giải thích để tệp dự đoán có nó mà không cần chạy lại. Chỉ nút *cuối cùng* ghi đáp án hướng tới người dùng mới bị giới hạn 30 từ (`REASON_THEN_ANSWER`, dùng cho Node 4 và Node 5): Node 1 (tóm tắt ca bệnh), Node 2 (tóm bằng chứng) và Node 3 (báo cáo lập luận) chủ ý *không* bị giới hạn độ dài, vì đó là đầu vào nội bộ cho Node 4/5 đọc, không phải đầu ra hướng tới người dùng — giới hạn từ ở đó sẽ chỉ làm cùn lập luận mà các nút này tồn tại để tạo ra.

4 Thiết lập thực nghiệm

4.1 Mô hình và prompt

Mô hình trả lời mặc định là **Qwen2.5** [8] (Ollama, có gọi công cụ): **qwen2.5:7b** làm mặc định cho CLI và **qwen2.5:14b** làm mặc định thư viện cho các con số được báo cáo. Mô hình 14B là điểm cân bằng năng lực/tốc độ trên phần cứng mục tiêu; **qwen2.5:32b** không cho thêm độ chính xác (0.66 so với 0.68) nhưng chậm gấp ~4 lần. Mọi vai chạy ở **hiệt độ 0**: ở nhiệt độ mặc định, chênh lệch V1 – V0 dao động giữa các lần chạy lớn hơn cả bản thân hiệu ứng, nên tính tất định là bắt buộc để McNemar ghép cặp có ý nghĩa. Các nhà cung cấp đám mây (**anthropic**:, **openai**:, **google**:, **openrouter**:) được hỗ trợ qua cùng một đặc tả **provider:model**.

System prompt của baseline (V0) và lượt người dùng cho câu trắc nghiệm, nguyên văn (giữ tiếng Anh vì đây là prompt vận hành thực tế):

```

System (roles.py:BASELINE):
You are an expert physician answering a medical board (USMLE)
multiple-choice question. Choose the single best answer, with a
brief justification.

User turn (qa/mcq.py:format_mcq):
{question}

A. {option 0}
B. {option 1}
C. {option 2}
D. {option 3}

Reason briefly, then on the LAST line write 'Answer: X' where X is
A, B, C, or D.

```

`parse_choice` trích đáp án từ **Answer: X** hoặc **answer is X** (không phân biệt hoa thường), dự phòng bằng chữ cái A|B|C|D độc lập đầu tiên; phản hồi không trích được trả về **None** và bị tính là *không hợp lệ*, không bao giờ bị tính nhầm thành câu trả lời sai. Tỷ lệ không hợp lệ đo được là **0.0%** xuyên suốt.

4.2 Cấu hình truy hồi

Bảng 2: Thiết lập truy hồi — một cấu hình duy nhất, dùng chung cho V1–V4 (kể cả bộ kiểm chứng của V3, vốn không còn nguồn truy hồi riêng).

Thiết lập	Giá trị
Ngữ liệu	MedMCQA: 182,822 câu thi đã giải (collection <code>knowledge_medmcqa_nomic</code>)
Kho vector Embedder	Chroma, cosine / HNSW, lưu trữ bền <code>nomic-embed-text</code> : ngữ cảnh 8192 token, nhúng trọn tình huống không bị cắt
Cái được nhúng Top- <i>k</i>	chỉ phần câu hỏi (đáp án/giải thích/chủ đề là metadata) 5, không cống (<code>threshold=0.0</code>)
Cổng độ tương đồng (ngưỡng cosine)	không dùng ở bất kỳ đâu trong hệ thống hiện tại

V1–V4 đều dùng **nomic-embed-text** vì Query encoder của các embedder bất đối xứng thường gặp (vd. MedCPT) cắt truy vấn ở khoảng **64 token**, quá ngắn cho một tình huống USMLE mà câu hỏi thực nằm ở cuối; `nomic` có ngữ cảnh 8192 token nên nhúng trọn cả tình huống. Thiết kế trước dùng một cổng độ tương đồng bằng số, hiệu chỉnh riêng theo embedder; cổng số đó nay không còn ở bất kỳ đường truy hồi nào trong hệ thống: việc quyết định có tin bằng chứng hay không chuyển hẳn sang bước chốt đáp án, dựa trên nhãn tin cậy HIGH/MEDIUM/LOW mà chính mô hình tự chấm (§3.2) — một cổng số học phía trên nhãn tự chấm đó là dư thừa. Bộ kiểm chứng của V3 từng có một ngữ liệu/embedder thứ hai riêng (MedRAG Textbooks, MedCPT) cho một công cụ tra cứu độc lập; ngữ liệu đó đã bị bỏ (§3.4), nên bảng trên là cấu hình truy hồi duy nhất còn lại trong hệ thống.

4.3 Phần cứng / thiết lập API

- **Phần cứng:** máy Mac Apple Silicon, 48 GB bộ nhớ hợp nhất, chạy Ollama cục bộ.
- **Seed ngẫu nhiên:** `bootstrap_ci(seed=0)`; sinh văn bản là greedy ở nhiệt độ 0 nên không áp dụng seed lấy mẫu.
- **Lưu ý khi đo độ trễ:** một câu khởi động tổng hợp được chạy trước (khởi động mô hình nguội sẽ tính cho câu chấm điểm đầu tiên ~ 1.9 s so với ~ 0.3 s ở trạng thái ổn định); các mô hình không phải mục tiêu bị dừng trước một lần chạy đo giờ để tránh tranh chấp GPU.
- **Dữ liệu:** `openlifescienceai/medqa` (train 10,178 / validation 1,272 / **test 1,273**), lưu cache cục bộ và nạp không cần mạng.

Tái lập một lần chạy.

```
HF_HUB_OFFLINE=1 PYTHONPATH=src .venv/bin/python -m agent_hospital -v V0 -n 150
```

5 Kết quả đánh giá

Chỉ số chính là độ chính xác, $\text{Độ chính xác} = (\text{số dự đoán đúng})/(\text{số câu hỏi})$; với đánh giá chính thức, mẫu số là toàn bộ tập test, 1273. Nhóm báo cáo thêm tỷ lệ phản hồi không hợp lệ, mức tăng độ chính xác $\text{Acc}(V3) - \text{Acc}(V0)$, Thắng/Thua/Hòa so với baseline, khoảng tin cậy bootstrap 95%, kiểm định ghép cặp McNemar, và độ trễ.

Trạng thái đo lường. Mục này chỉ báo cáo **lần chạy chính thức trên toàn bộ tập test** ($n=1273$, qwen2.5:7b, nhiệt độ 0). Các số ở quy mô nhỏ trước đây ($n=80$, 14b) đã được gỡ bỏ để tránh nhầm lẫn. Hiện mới có V0; V1–V4 đang chạy trên cùng tập câu, nên các chỉ số ghép cặp (mức tăng, Thắng/Thua/Hòa, McNemar) sẽ được điền khi đủ dữ liệu.

5.1 Bảng chính (main board): test đầy đủ ($n=1273$, qwen2.5:7b)

Lần chạy chính thức trên *toàn bộ* tập test dùng **qwen2.5:7b** (nhẹ hơn 14b để chạy hết 1273 câu). Hiện mới có V0; các biến thể còn lại đang được chạy trên cùng tập câu để tính các chỉ số ghép cặp. Token/câu (đầu vào + đầu ra) là thước đo chi phí cho mô hình cục bộ. V1 **thấp hơn V0**

Bảng 3: Kết quả chính thức: MedQA test đầy đủ, qwen2.5:7b, nhiệt độ 0, $n=1273$.

Biến thể	Độ chính xác	KTC bootstrap 95%	Không hợp lệ	Độ trễ/câu
V0: LLM trực tiếp	0.615 (783/1273)	[0.588, 0.641]	0.0%	0.97 s
V1: RAG-only (agentic)	0.573 (730/1273)	[0.546, 0.602]	3.85%	3.98 s
V2: Hội đồng + kiểm chứng (agentic)	0.608 (774/1273)	[0.581, 0.634]	0.0%	23.84 s
V3–V4	[CẦN LÀM: đang chạy trên test đầy đủ, qwen2.5:7b]			

(-4.2 điểm), tốn gấp ~ 5 lần token và ~ 4 lần độ trễ, và có tỷ lệ phản hồi không hợp lệ cao hơn hẳn (3.85% so với 0%) — vòng lặp gọi công cụ đôi khi không hội tụ về một chữ cái. V2 **gần bằng V0** (-0.7 điểm, khoảng tin cậy chồng lấn phần lớn), tỷ lệ không hợp lệ trở về 0% (hội đồng + bộ kiểm chứng loại bỏ được vấn đề không hội tụ của V1), nhưng chi phí rất cao: ~ 25 lần độ trễ và ~ 16 lần token so với V0. Kết quả này nhất quán với mọi lần đo trước đó trong dự án: retrieval không giúp ích trên MedQA ở mô hình này; cấu trúc đa tác tử (V2) phục hồi phần lớn khoảng cách của V1 nhưng chưa vượt qua V0. **[CẦN LÀM: Bản ghi từng câu của V0 (dùng để chấm**

điểm) đã mất do dọn dẹp thư mục tạm giữa chừng; số của V0 trên chỉ còn ở dạng tổng hợp (accuracy/KTC/độ trễ/token), nên Bảng 4 (so sánh ghép cặp, Thắng/Thua/Hòa, McNemar) cho V1/V2 vs V0 hiện chưa tính được — cần chạy lại V0 để có bản ghi ghép cặp nếu muốn số liệu đó.]

5.2 So sánh với baseline và khảo sát ablation (so sánh ghép cặp)

Bảng 4: So sánh ghép cặp trên test đầy đủ (n=1273, qwen2.5:7b). Tính từ bản ghi từng câu của V0 và mỗi biến thể.

Cặp so sánh	Mức tăng độ chính xác	Thắng/Thua/Hòa	McNemar
V1 vs V0 (chỉ độ chính xác)	−4.2 điểm	[CẦN LÀM: cần bản ghi ghép cặp c	
V2 vs V0 (chỉ độ chính xác)	−0.7 điểm	[CẦN LÀM: cần bản ghi ghép cặp c	
V3 vs V0 (mức tăng Acc(V3) − Acc(V0))		[CẦN LÀM: đang chạy]	
V3 vs V4 (hiệu ứng kiểm chứng)		[CẦN LÀM: đang chạy]	

5.3 Phân tích Thắng/Thua/Hòa

Cột T/T/H đếm theo từng câu: biến thể sửa được lỗi của baseline (thắng), làm hỏng một câu baseline vốn đúng (thua), hay trùng khớp (hòa) — đây là đầu vào cho kiểm định McNemar ở Bảng 4. [CẦN LÀM: V1/V2 vs V0 chưa tính được: bản ghi từng câu của V0 đã mất (xem ghi chú ở Bảng 3). Điền phần còn lại từ bản ghi test đầy đủ khi V3–V4 chạy xong.]

5.4 Chi phí và độ trễ

Chi phí đo bằng độ trễ và số token (đầu vào + đầu ra) cho mô hình cục bộ (không có phí API). V0 trên test đầy đủ: 0.97s/câu, 329 token/câu, tỷ lệ phản hồi không hợp lệ 0%. V1 (agentic): 3.98s/câu (~4× V0), 1668 token/câu (~5× V0), không hợp lệ 3.85% — vòng lặp gọi công cụ (tìm rồi trả lời) sinh nhiều token hơn hẳn một lượt trả lời trực tiếp, và thỉnh thoảng không hội tụ về một chữ cái. V2 (hội đồng 3 chuyên gia + bác sĩ chính + bộ kiểm chứng, agentic): 23.84s/câu (~25× V0), 5304 token/câu (~16× V0), không hợp lệ 0% — nhiều tác tử tuần tự (không chạy song song) mỗi tác tử có thể gọi công cụ, nên chi phí cộng dồn nhanh; xem §3 về khả năng chạy song song 3 chuyên gia để giảm độ trễ. [CẦN LÀM: Bổ sung độ trễ và token của V3–V4 khi chạy xong; V3 (giống V2 nhưng thêm bộ nhớ ngắn hạn) kỳ vọng cao hơn 23.84s/câu của V2; V4 dùng thiết kế truy hồi cũ (graph-invoked), chưa đo lại ở qwen2.5:7b.]

6 Phân tích lỗi

[CẦN LÀM: Mục này cần tệp dự đoán từng câu (§9) để hoàn thiện; dưới đây là mô tả định tính hiện tại từ các lần chạy ghép cặp.]

Tỷ lệ không hợp lệ là 0.0% trên mọi biến thể đã đo, nên mọi lỗi đều là chọn sai lựa chọn thực sự, không phải lỗi trích xuất. Hai mẫu hình hiện rõ trong dữ liệu ghép cặp:

- **Câu trung tính với truy hỏi.** Trên đa số câu, truy hỏi hầu như không đổi đáp án, nhất quán với việc MedQA chủ yếu đòi hỏi lập luận: mô hình vốn đã nắm dữ kiện, nên thêm dữ kiện không đổi đáp án.
- **Câu nhảy với lập luận.** Các câu mà V2 sửa được so với V1 là những câu mà việc cấu trúc hoá phân tích (dấu hiệu $\rightarrow$ câu hỏi hỏi gì $\rightarrow$ từng lựa chọn) làm đổi quyết định, dù bằng chứng truy hỏi giống hệt. Điều này khu trú mức tăng vào khâu lập luận.

Bảng 5: Bảng phân tích lỗi (khung; sẽ điền từ tệp dự đoán).

Loại lỗi	Mã câu ví dụ	Ghi chú
Lỗi lập luận (tích hợp sai)	[CẦN LÀM:]	truy hỏi đúng dữ kiện nhưng chọn sai lựa chọn
Bị phương án nhiễu đánh lừa	[CẦN LÀM:]	tự tin chọn một phương án hợp lý nhưng sai
Truy hỏi trượt / bằng chứng lạc đề	[CẦN LÀM:]	cổng nhận một đoạn không phân biệt được
Thiếu tri thức	[CẦN LÀM:]	dữ kiện thực sự vắng ở cả mô hình lẫn ngữ liệu

7 Thảo luận

Bài học nổi bật là: **trên MedQA, đòn bẫy là lập luận, không phải truy hỏi.** Đây không phải lỗi tinh chỉnh: Nhóm đã thử hai ngữ liệu (sách giáo khoa -3.7 điểm; MedMCQA $+0.5$ điểm, không ý nghĩa), đã hiệu chỉnh cổng, và đã rút gọn truy vấn, mà truy hỏi vẫn không làm thay đổi độ chính xác. Y văn cũng ghi nhận điều tương tự: mức tăng của RAG không đồng đều giữa các bộ dữ liệu: mạnh trên các bộ nặng về tra cứu (PubMedQA, BioASQ) và yếu trên các bộ nặng về lập luận/trí nhớ như MedQA, vì một mô hình đủ mạnh vốn đã nắm sẵn tri thức trong sách giáo khoa và phần việc còn lại là *tích hợp*. Kết quả V2 của Nhóm là mặt tích cực của câu chuyện này: một cách tách hai tác tử phân biệt phân tích lâm sàng có cấu trúc với quyết định cuối cùng tạo ra mức cải thiện có ý nghĩa thống kê đầu tiên, và làm được điều đó trên *cùng* bằng chứng truy hỏi mà V1 đã có. Điều đó chỉ hướng đòn bẫy tiếp theo vào cấu trúc lập luận (hội đồng, bác sĩ chính, bộ kiểm chứng), đúng là thứ V3/V4 được dựng để kiểm tra.

8 Hạn chế

- **Quy mô.** Hiện mới có V0 trên test đầy đủ ($n=1273$, qwen2.5:7b); V1–V4 đang chạy, nên các so sánh ghép cặp còn trống cho tới khi đủ dữ liệu.
- **Thiếu thành phần.** Bộ nhớ dài hạn (§4.5) chưa được xây; bộ nhớ ngắn hạn (§4.4) chỉ mới ở dạng hẹp (đọc trạng thái đồ thị dùng chung, chỉ phục vụ bộ kiểm chứng), nên hệ thống hiện tại vẫn chưa phải “hệ đầy đủ” theo đề bài.
- **Chưa có chỉ số token/chi phí.** Mới chỉ ghi độ trễ; chưa có chi phí token và tiền.
- **Chưa có tệp dự đoán / công cụ phân tích lỗi** (§9), nên phân tích lỗi còn định tính.
- **Số liệu cũ đã bị thay thế.** Việc thêm giải thích ≤ 30 từ, rồi đột tái thiết kế V2–V4 và luật tin cậy bất đối xứng, đã lần lượt làm đổi mọi prompt, nên các phép đo trước mỗi lần đổi không còn áp dụng cho thiết kế hiện tại và đang được đo lại.
- **Cổng do mô hình tự chấm chưa được kiểm định.** Cổng độ tương đồng bằng số đã được thay bằng nhãn tin cậy tự chấm của chính mô hình (§3.2); độ tin cậy của chính nhãn tự chấm đó (so với một ngưỡng số khách quan) chưa được đo.

- **Tính tắt định trên đám mây.** Nhiệt độ 0 được đặt, nhưng các API lưu trữ không bảo đảm tính tái lập như giải mã greedy cục bộ; cổng golden được ghim vào một mô hình cục bộ.

9 Kết luận

Nhóm đã xây một hệ đa tác tử y khoa điều khiển bằng cấu hình dưới dạng một thang biến thể năm bậc (ablation ladder) trên MedQA-USMLE, trong đó mỗi bậc khác nhau đúng một thành phần và được chấm bởi một bộ đo ghép cặp, có nhận thức về ý nghĩa thống kê. Bằng chứng hiện tại rõ ràng ở điểm cốt lõi: riêng truy hồi không cải thiện độ chính xác MedQA, trong khi một cách tách lập luận đa tác tử thì có (kết quả định lượng đang được đo lại trên test đầy đủ). Các bước tiếp theo trước mắt là (i) thống nhất ngữ liệu truy hồi giữa các bậc, (ii) chạy toàn bộ tập test chính thức ($n=1273$), (iii) đo V3/V4 ở quy mô lớn và tách riêng hiệu ứng bộ kiểm chứng ($V3 - V4$), và (iv) thêm bộ nhớ, tệp dự đoán, và theo dõi chi phí token để khép các khoảng trống còn lại so với đề bài. Hệ thống được thiết kế sao cho mỗi việc trên là một thay đổi cấu hình hoặc bộ đo chứ không phải viết lại.

A Danh mục kiểm tra tái lập

Mục	Giá trị
Mô hình	<code>qwen2.5:14b</code> (mặc định thư viện), <code>qwen2.5:7b</code> (mặc định CLI)
Nhiệt độ	0 (mọi vai)
Prompt	nguyên văn trong <code>roles.py</code> ; baseline xem §4
Truy hồi	<code>embedder nomic-embed-text</code> , Chroma cosine/HNSW, $\text{top-}k=5$, không cổng
Seed ngẫu nhiên	<code>bootstrap_ci(seed=0)</code> ; còn lại là giải mã greedy
Phần cứng	máy Mac Apple Silicon, 48 GB, Ollama cục bộ
Dữ liệu	<code>openlifescienceai/medqa</code> (test $n=1273$), cache ngoại tuyến
Điểm vào	<code>build_variant(id, model=, temperature=0, **overrides)</code>

Tài liệu tham khảo

- [1] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. volume 11, page 6421, 2021.
- [2] Qiao Jin, Won Kim, Qingyu Chen, Donald C Comeau, Lana Yeganova, W John Wilbur, and Zhiyong Lu. Medcpt: Contrastive pre-trained transformers with large-scale pubmed search logs for zero-shot biomedical information retrieval. *Bioinformatics*, 39(11), 2023.
- [3] Yubin Kim, Chanwoo Park, Hyewon Jeong, Yik Siu Chan, Xuhai Xu, Daniel McDuff, Hyeon-hoon Lee, Marzyeh Ghassemi, Cynthia Breazeal, and Hae Won Park. Mdagents: An adaptive collaboration of llms for medical decision-making. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2404.15155.
- [4] LangChain. Langgraph: Building stateful, multi-actor applications with llms. <https://langchain-ai.github.io/langgraph/>, 2024.

- [5] Junkai Li, Yunghwei Lai, Weitao Li, Jingyi Ren, Meng Zhang, Xinhui Kang, Siyu Wang, Peng Li, Ya-Qin Zhang, Weizhi Ma, and Yang Liu. Agent hospital: A simulacrum of hospital with evolvable medical agents. *arXiv preprint arXiv:2405.02957*, 2024.
- [6] Harsha Nori, Yin Tat Lee, Sheng Zhang, Dean Carignan, Richard Edgar, et al. Can generalist foundation models outcompete special-purpose tuning? case study in medicine. *arXiv preprint arXiv:2311.16452*, 2023.
- [7] Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Conference on Health, Inference, and Learning (CHIL)*, 2022.
- [8] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.
- [9] Xiangru Tang, Anni Zou, Zhuosheng Zhang, Ziming Li, Yilun Zhao, Xingyao Zhang, Arman Cohan, and Mark Gerstein. Medagents: Large language models as collaborators for zero-shot medical reasoning. In *Findings of the Association for Computational Linguistics (ACL Findings)*, 2024. arXiv:2311.10537.
- [10] Guangzhi Xiong, Qiao Jin, Zhiyong Lu, and Aidong Zhang. Benchmarking retrieval-augmented generation for medicine. *Findings of the Association for Computational Linguistics (ACL)*, 2024.